

Herramienta seleccionada: Postman
Fecha de entrega: 17/09/2026

1. Investigación teórica

1.1 Fundamentos del testing de APIs

Una API REST expone recursos a través de una URL y responde a un conjunto reducido de verbos HTTP: GET para consultar, POST para crear, PUT/PATCH para actualizar y DELETE para eliminar. Cada respuesta incluye un código de estado numérico que indica el resultado de la operación: los códigos 2xx señalan éxito, los 3xx redirecciones, los 4xx errores originados en el cliente (por ejemplo, un dato mal enviado) y los 5xx errores del servidor (Mozilla Developer Network, s.f.). Probar a nivel de API, en lugar de hacerlo únicamente a través de la interfaz gráfica, permite validar la lógica de negocio de forma más rápida y estable, porque no depende de que se hayan renderizado pantallas ni de la velocidad de un navegador.

Esta idea se relaciona con el concepto de "pirámide de testing", popularizado por Mike Cohn y desarrollado en detalle por Fowler y Vocke (2018): las pruebas unitarias y de API deberían ser la base de la estrategia porque son rápidas y estables, mientras que las pruebas de interfaz gráfica (más lentas y frágiles) deberían usarse con moderación, como una segunda línea de defensa.

1.2 Postman: arquitectura y componentes

Postman organiza el trabajo en collections (conjuntos de requests agrupados y versionables) y environments (conjuntos de variables — URLs base, tokens, credenciales — que cambian según el contexto: desarrollo, staging o producción). Usar variables como {{base_url}} en lugar de escribir la URL completa en cada request permite cambiar de entorno sin reescribir nada (Postman, Inc., s.f.-a).

1.3 Test scripts y assertions

Postman permite adjuntar scripts en JavaScript a cada request, que se ejecutan después de recibir la respuesta ("Post-response scripts"). El método pm.test(nombre, función) define una prueba con un nombre descriptivo y una función que contiene la validación; dentro de esa función se usa pm.expect() (basado en la librería de aserciones Chai) o los atajos de pm.response.to.have.* para comprobar, por ejemplo, que el código de estado sea 200 o que el cuerpo de la respuesta contenga determinado campo (Postman, Inc., s.f.-b, s.f.-c).

1.4 Newman: ejecución por línea de comandos

Newman es el ejecutor de línea de comandos oficial de Postman: permite correr una collection exportada como archivo JSON sin abrir la aplicación gráfica, lo que lo hace apto para integrarse en pipelines de integración continua. Se instala vía npm install -g newman y se ejecuta con newman run coleccion.json -e entorno.json, generando reportes exportables (Postman Labs, s.f.).

1.5 Postman en flujos de trabajo profesionales y seguridad de APIs

Más allá de la ejecución manual, Postman se usa en equipos para compartir colecciones, documentar automáticamente los endpoints y monitorear APIs en producción. Esto se enmarca en una preocupación mayor: la seguridad de las APIs. El proyecto OWASP API Security Top 10 identifica los riesgos más frecuentes en APIs, como la autorización rota a nivel de objeto o la falta de límites de recursos y de tasa de peticiones, y sirve como checklist de referencia para que un plan de pruebas no se limite a validar que "la API responde", sino también que responde de forma segura (OWASP API Security Project, 2023).

Frameworks profesionales como REST-assured (para Java) muestran que los mismos conceptos de testing de API — dar una petición, validar una respuesta — se repiten más allá de Postman, lo que confirma que las habilidades adquiridas con esta herramienta son transferibles (Baeldung, s.f.).

2. Referencias (formato APA)

Baeldung. (s.f.). A guide to REST-assured. Baeldung. https://www.baeldung.com/category/testing/tag/rest-assured

Fowler, M., & Vocke, H. (2018, 26 de febrero). The practical test pyramid. martinfowler.com. https://martinfowler.com/articles/practical-test-pyramid.html

Mozilla Developer Network. (s.f.). HTTP response status codes. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

OWASP API Security Project. (2023). OWASP API Security Top 10 — 2023 edition. OWASP Foundation. https://owasp.org/API-Security/

Postman Labs. (s.f.). Newman: Command-line collection runner for Postman [Repositorio de software]. GitHub. https://github.com/postmanlabs/newman

Postman, Inc. (s.f.-a). Collections overview. Postman Learning Center. https://learning.postman.com/docs/collections/collections-overview/

Postman, Inc. (s.f.-b). Writing tests and assertions in scripts. Postman Learning Center. https://learning.postman.com/docs/tests-and-scripts/write-scripts/postman-sandbox-reference/pm-test-expect/

Postman, Inc. (s.f.-c). Postman test script examples. Postman Learning Center. https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-examples/

Spring Framework. (s.f.). *Testing in Spring Boot*. Spring Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/testing.html

Carnegie Mellon University Software Engineering Institute. (2020). *10 recommended practices for achieving agile at scale*. https://www.sei.cmu.edu/blog/10-recommended-practices-for-achieving-agile-at-scale/

3. Ficha del sistema bajo prueba

¿Qué es el sistema que van a probar?
GESTION-DE-ALUMNOS: una API REST desarrollada en Java con Spring Boot para la gestión de alumnos (operaciones CRUD sobre alumnos, cursos y entidades relacionadas), con persistencia en base de datos MySQL.

¿Por qué eligieron ese sistema?
Es un proyecto propio del grupo, ya desarrollado en el marco de otra materia, lo que garantiza acceso total al código fuente, conocimiento previo de su arquitectura y la posibilidad de correrlo en localhost sin exponer datos de terceros.

¿Cómo verificaron que tienen autorización para probarlo?
Al ser autores del propio repositorio (código escrito por integrantes de Grupo 5), la autorización es inherente: se prueba código propio, en un entorno local, sin acceder a sistemas de terceros ni a datos de producción ajenos.

¿Cuáles son sus características principales?

- Lenguaje: Java
- Framework: Spring Boot
- Arquitectura: REST (controller–service–repository)
- Persistencia: MySQL (con H2 como alternativa para pruebas)
- Complejidad: media — múltiples endpoints CRUD con validaciones de datos